MULTI-CLOUD ARCHITECTURE 101

AWS + Azure + GCP: what does a real multi-cloud architecture look like?

Not three copies of everything. Workloads are placed per cloud. Shared layers run across all of them.

① IDENTITY

One identity provider, federated SSO into every cloud. No separate user lists.

AWS	Azure	GCP
Workload A	Workload B	Workload C
Its data	Its data	Its data

Place each workload where it fits best, and keep its data close to it

② NETWORK

Private connectivity, non-overlapping CIDRs, and a clear view of where egress costs money.

③ OBSERVABILITY

One telemetry pipeline (e.g. OpenTelemetry) feeding one place to look.

④ DELIVERY & POLICY

One CI/CD workflow and one policy/tagging standard, applied to every cloud.

01 · THE HOOK

"We're multi-cloud" can mean two very different things

Picture one: an architecture

Workloads deliberately placed in AWS, Azure and GCP, held together by shared identity, networking, observability and delivery standards.

Picture two: an accident

Three clouds that grew separately, each with its own users, pipelines, dashboards and naming. Nobody designed it.

The clouds are not the architecture. The layers that run across them are.

02 · THE PROBLEM

What breaks when no one owns the layers in between

- → **Access drift.** Users and roles managed in three places means a leaver eventually gets missed in one of them.
- → **Hidden network costs.** Cross-cloud traffic adds latency, and egress charges show up weeks after the design decision.
- → **Slow incidents.** Symptom in one cloud, cause in another, with three consoles and three log formats.
- → **Inconsistent delivery.** Three pipelines means three definitions of "production-ready".

03 · THE ARCHITECTURE

Shared layers across, workloads placed per cloud

Identity: one IdP, federated SSO everywhere

AWS
Workload A + its data

Azure
Workload B + its data

GCP
Workload C + its data

Network: one IP plan, private links, controlled egress

Observability: one telemetry pipeline, one place to look

Delivery & policy: one CI/CD workflow, one tagging standard

Same layers, different services

	AWS	Azure	GCP
Workforce SSO	IAM Identity Center	Microsoft Entra ID	Cloud Identity / Workforce Identity Federation
Workload identity	IAM roles	Managed identities	Service accounts
VPN	Site-to-Site VPN	VPN Gateway	Cloud VPN (HA VPN)
Private link	Direct Connect	ExpressRoute	Cloud Interconnect
Metadata	Tags	Tags	Labels (lowercase)

> **Tip:** check current provider docs before designing. Service names and capabilities change.

Agree on these before the first VPC exists

A non-overlapping IP plan

```
# example: one /16 per environment, never reused
on-prem   10.0.0.0/16
aws       10.10.0.0/16
azure     10.20.0.0/16
gcp       10.30.0.0/16
reserved  10.40.0.0/13   # growth
```

One tagging standard that's valid everywhere

```
# lowercase keys + values satisfy GCP label rules
owner       = "team-payments"
env         = "prod"
cost-center = "cc-1234"
data-class  = "confidential"
```

05 · THE TRADE-OFFS

Multi-cloud is a tool, not a goal

- → **Skills:** every cloud adds an IAM model, a network model and new failure modes for on-call.
- → **Lowest common denominator:** abstracting everything can cost you each cloud's best managed services.
- → **Data gravity:** moving data between clouds costs money and time.
- → **New dependencies:** cross-cloud links and federation become production systems in their own right.

Worth it when: regulation or data residency requires it, one cloud has a service you genuinely need, vendor-risk policy demands it, or an acquisition brings another cloud with it.

06 · THE TAKEAWAY

Multi-cloud readiness checklist

- ☐ One IdP with SSO into every cloud
- ☐ Workload identity, not long-lived keys
- ☐ A written, non-overlapping IP plan
- ☐ Cross-cloud traffic paths and egress costs documented
- ☐ A documented reason for every workload's placement
- ☐ One telemetry pipeline, one incident starting point
- ☐ One CI/CD workflow and tagging standard, enforced by policy

07 · YOUR TURN

What would you place in each cloud, and why?

Share your design in the comments. The best answers come from engineers who've lived with the trade-offs.

Follow Deji Ajishe

for the rest of this 30-part multi-cloud DevOps series

♻️ Repost or share this with an engineer on your team

linkedin.com/in/deji-ajishe-64a35722a